DHBW Heilbronn

Software Engineering II.1 – Block 2

Requirements Engineering

Vom Projektauftrag zu prüfbaren Anforderungen

Software Engineering II.1 | WS 2026/27 | Dr. rer. nat. Ana Nicolaescu

Wo stehen wir im Kurs?

Block 1	**Block 2**	Block 3	Block 4	Block 5	Block 6	Block 7
4 UE	4 UE	4 UE	4 UE	4 UE	4 UE	4 UE
Einführung & Vorgehensmodelle	Requirements Engineering	Entwurf & Entwurfsmuster	Implementierung & Versionsmanagement	Testen (Blackbox)	Whitebox, Wartung & Change Request	Management & Abschlusspräsentation

Durchgängiges Gruppenprojekt „Photo Album“: jeder Block liefert ein prüfbares Artefakt ins Repository

Block 1	Block 2	Block 3	Block 4	Block 5	Block 6	Block 7
Teams, Repo, Vorgehen REQUIREMENTS.md	REQUIREMENTS.md Use Cases · AK · Prototyp	ARCHITECTURE.md Klassen- · Sequenzdiagramm	Build · Git · PR Review · CI grün	TESTING.md Unit/Integr. · Blackbox	Coverage · Refactoring Change Request	RISK_ANALYSIS.md Präsentation

Heute entsteht die prüfbare Projektbasis

- Stakeholder und Systemgrenze des Photo Albums festlegen
- Funktionale Anforderungen aus Use Cases ableiten (nicht neu erfinden!)
- Anforderungen nach den SOPHIST-Regeln prüfen und mit Satzschablonen formulieren
- Akzeptanzkriterien schreiben, die später zu Testfällen werden
- Anforderungen an KI-Funktionen, Agenten, Skills und MCP-Tools verstehen
- Eigene Artefakte im Peer Review prüfen lassen

Projektergebnis dieses Blocks
REQUIREMENTS.md v1: 6–10 FA, 3–5 NFA
ausgearbeitete Use Cases (Beschreibung)
Akzeptanzkriterien (Given/When/Then)
Skizze / Klick-Prototyp der UI
Eintrag in AI_USAGE.md

Ablauf heute (240 Minuten)

00–10	Einstieg: Was bedeutet „schnell“? · Ziel des Blocks	**Übung**
10–30	Stakeholder und Systemgrenze des Photo Albums	**Übung**
30–65	Vom Use Case zur funktionalen Anforderung	**Übung**
65–105	SOPHIST-Regeln und Satzschablonen	**Übung**
105–120	Pause	**Pause**
120–145	Akzeptanzkriterien: wofür und wie	**Übung**
145–180	Requirement Clinic und Peer Review	**Teamarbeit**
180–210	KI-Perspektive: Anforderungen an KI, Agenten, Skills, MCP	**Übung**
210–225	Auftraggeber-Sprechstunde	**Teamarbeit**
225–240	Projektcheckpoint · Exit Ticket · Hausaufgabe	

Einstieg: Was bedeutet „schnell“?

Übung

Aus einem Teamprotokoll nach Block 1: „Die App soll Fotoalben schnell anzeigen.“

3 min

Einzeln, dann Team

AUFTRAG

1. 1 Minute allein: Schreiben Sie eine Version, die ein Tester eindeutig prüfen könnte.
2. 2 Minuten im Team: Einigen Sie sich auf eine Formulierung – notieren Sie die Annahmen, die Sie treffen mussten.

ERGEBNIS Eine prüfbare Formulierung pro Team auf dem Whiteboard/Miro.

DANACH IM PLENUM Zwei Teams lesen vor.

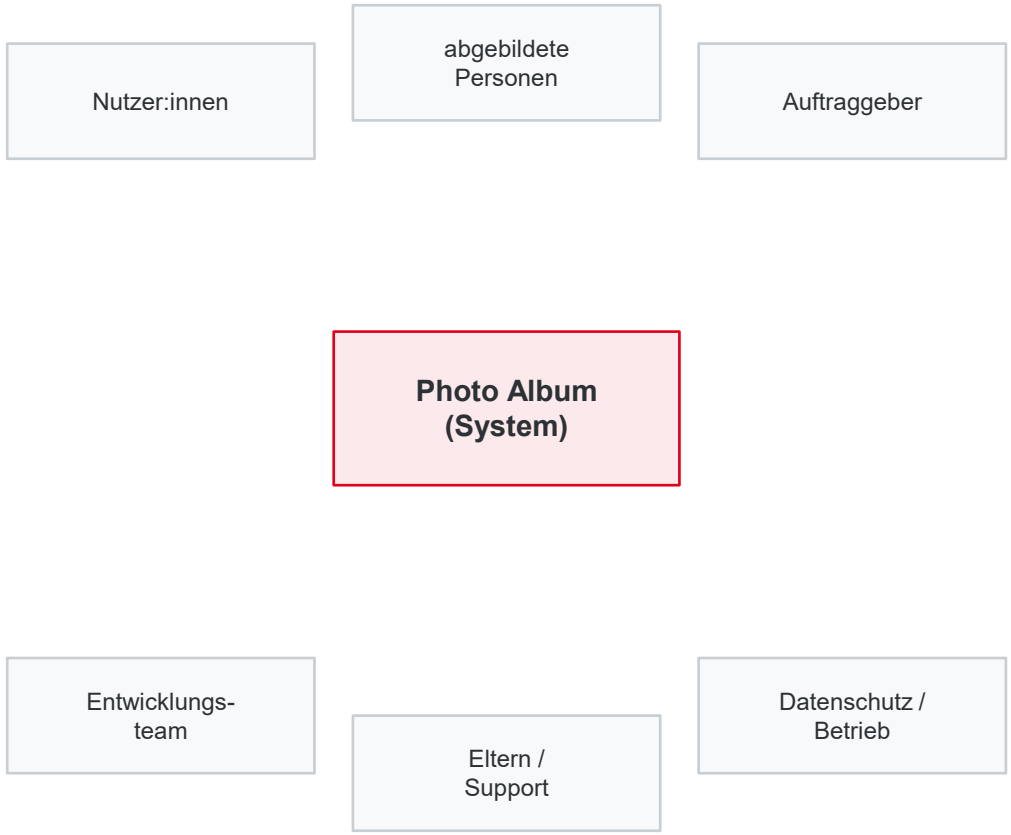
1

Stakeholder und Systemgrenze

Wer stellt Anforderungen – und wo endet unser System?

Stakeholder beeinflussen die Anforderungen

- **Primäre Nutzer:innen (Jugendliche, junge Erwachsene)**
  - Alben schnell anlegen, Fotos sortieren, mobil bedienen
- **Auftraggeber (das Unternehmen)**
  - Nutzen, Kosten, Termin, spätere Erweiterbarkeit
- **Entwicklungsteam**
  - Umsetzbarkeit, Wartbarkeit, Testbarkeit
- **Betrieb, Datenschutz, Recht**
  - Fotos sind personenbezogene Daten – Löschung, Zugriff, Nachweis
- **Indirekte Stakeholder**
  - abgebildete Personen, Eltern minderjähriger Nutzer:innen, Support



Nähe zum System ≈ Einfluss auf die Anforderungen

Übung: Stakeholder-Matrix für Ihr Photo Album

Übung

Legen Sie in REQUIREMENTS.md den Abschnitt „Stakeholder und Systemgrenze“ an.

8 min

Team

AUFTRAG

1. Nennen Sie mindestens 4 Stakeholder mit Interesse und Einfluss (hoch/mittel/niedrig).
2. Formulieren Sie pro Stakeholder eine Anforderung, die nur dieser Stakeholder stellt.
3. Zeichnen Sie das Kontextdiagramm: Was ist drin, was ist Schnittstelle, was ist bewusst draußen?

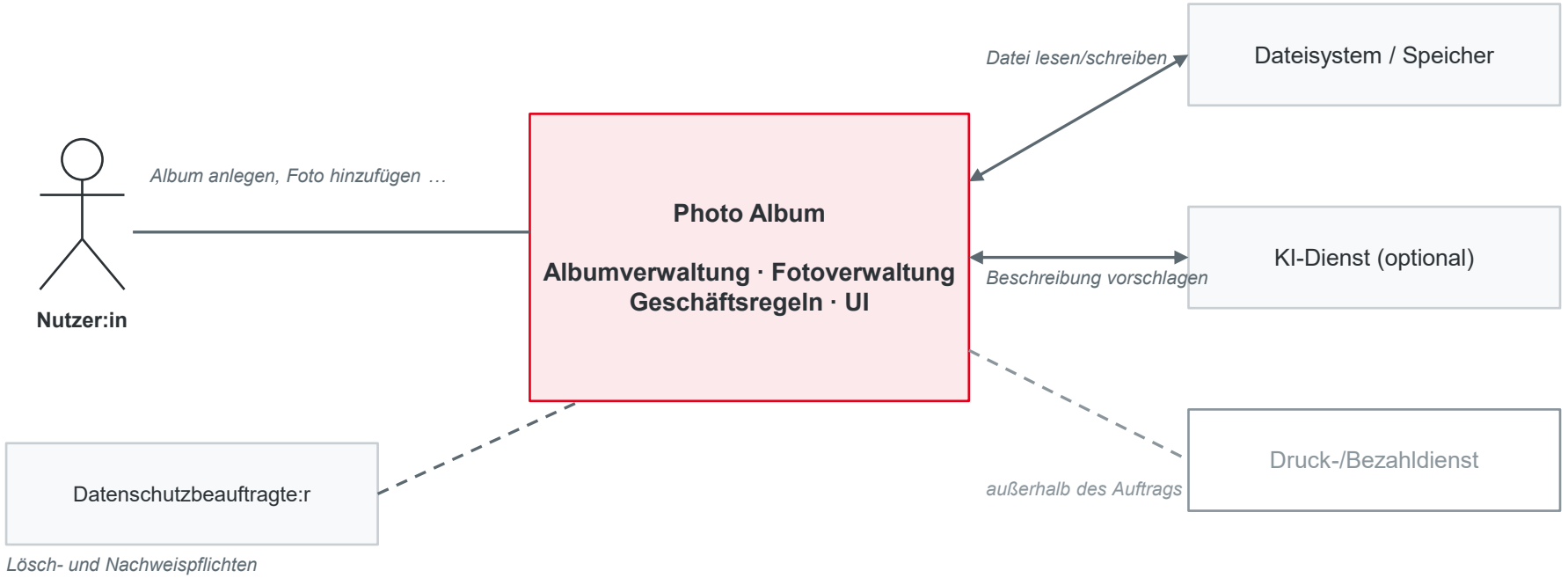
ERGEBNIS Tabelle + Kontextdiagramm (Foto/Miro) committed als erster Abschnitt von REQUIREMENTS.md.

MATERIAL Vorlage REQUIREMENTS.md

DANACH IM PLENUM Ein Team zeigt sein Kontextdiagramm; ein anderes Team nennt einen Stakeholder, der fehlt.

Systemgrenze und Kontextdiagramm

Im System: fachliche Funktionen, Regeln, Oberfläche. Außerhalb: Menschen, Nachbarsysteme, Infrastruktur. Jede Kante ist eine Schnittstelle mit eigenen Anforderungen und Fehlerfällen.



Merksatz: Die Systemgrenze schützt vor unbemerkter Scope-Erweiterung – was draußen ist, wird nicht gebaut, aber als Schnittstelle spezifiziert.

2

Vom Use Case zur funktionalen Anforderung

Use Cases kennen Sie aus SAE – heute leiten wir daraus prüfbare Anforderungen ab.

Was Sie aus SAE mitbringen – und was heute dazukommt

Bereits gelernt (Systemanalyse und -entwurf)

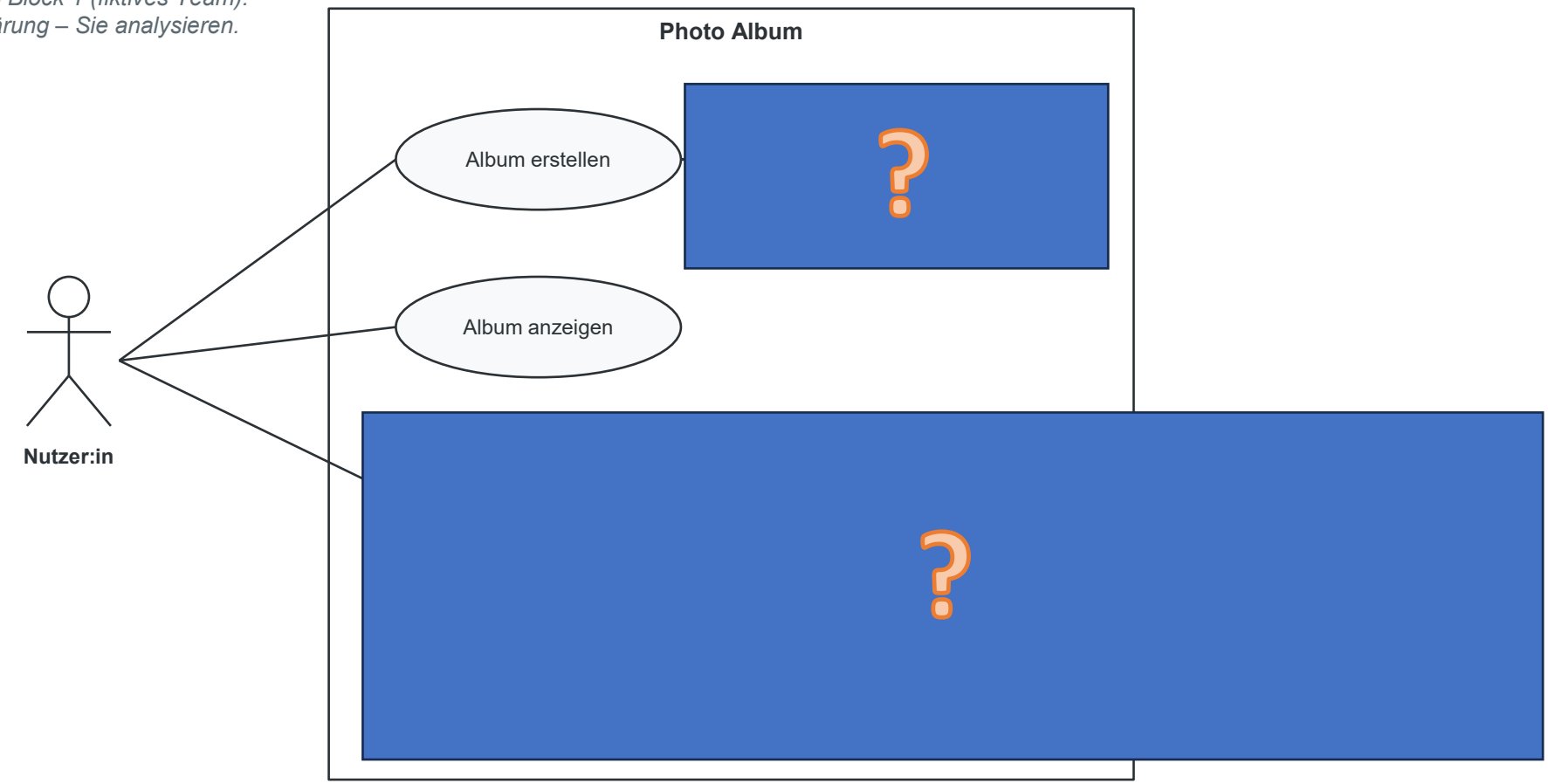
- Use-Case-Diagramm: Akteure, Systemgrenze, include, extend, Generalisierung
- Use-Case-Beschreibung: Vor-/Nachbedingung, Standard- und Alternativabläufe
- Aktivitätsdiagramme für fachliche Abläufe
- Domänen-Klassendiagramm aus Anforderungen
- Qualitätsregeln: Ziel, fachlich, Nutzen, Verb

Neu in Block 2

- Aus Use-Case-Schritten einzelne, nummerierte Anforderungen ableiten
- Funktional vs. nichtfunktional sauber trennen und NFA messbar machen
- Sprachliche Qualität: SOPHIST-Regeln und Satzschablonen
- Akzeptanzkriterien als Brücke zum Test (Block 5)
- Anforderungen an KI-Komponenten, Agenten, Skills, MCP-Tools

Use-Case-Diagramm Photo Album – was fehlt?

Stand nach Block 1 (fiktives Team).
Keine Erklärung – Sie analysieren.



Übung: Use-Case-Analyse und Ableitung von Anforderungen

Übung

Prüfen Sie das Diagramm gegen die Pflichtfunktionalität der Projektbeschreibung (Abschnitt 4.1) und die SAE-Qualitätsregeln.

15 min

Team

AUFTRAG

1. Welche Use Cases fehlen? Welche Beziehung ist fachlich fragwürdig? Welcher Akteur ist falsch klassifiziert?

2. Wählen Sie „Foto hinzufügen“: Schreiben Sie Standardablauf (max. 6 Schritte) und 2 Alternativabläufe.

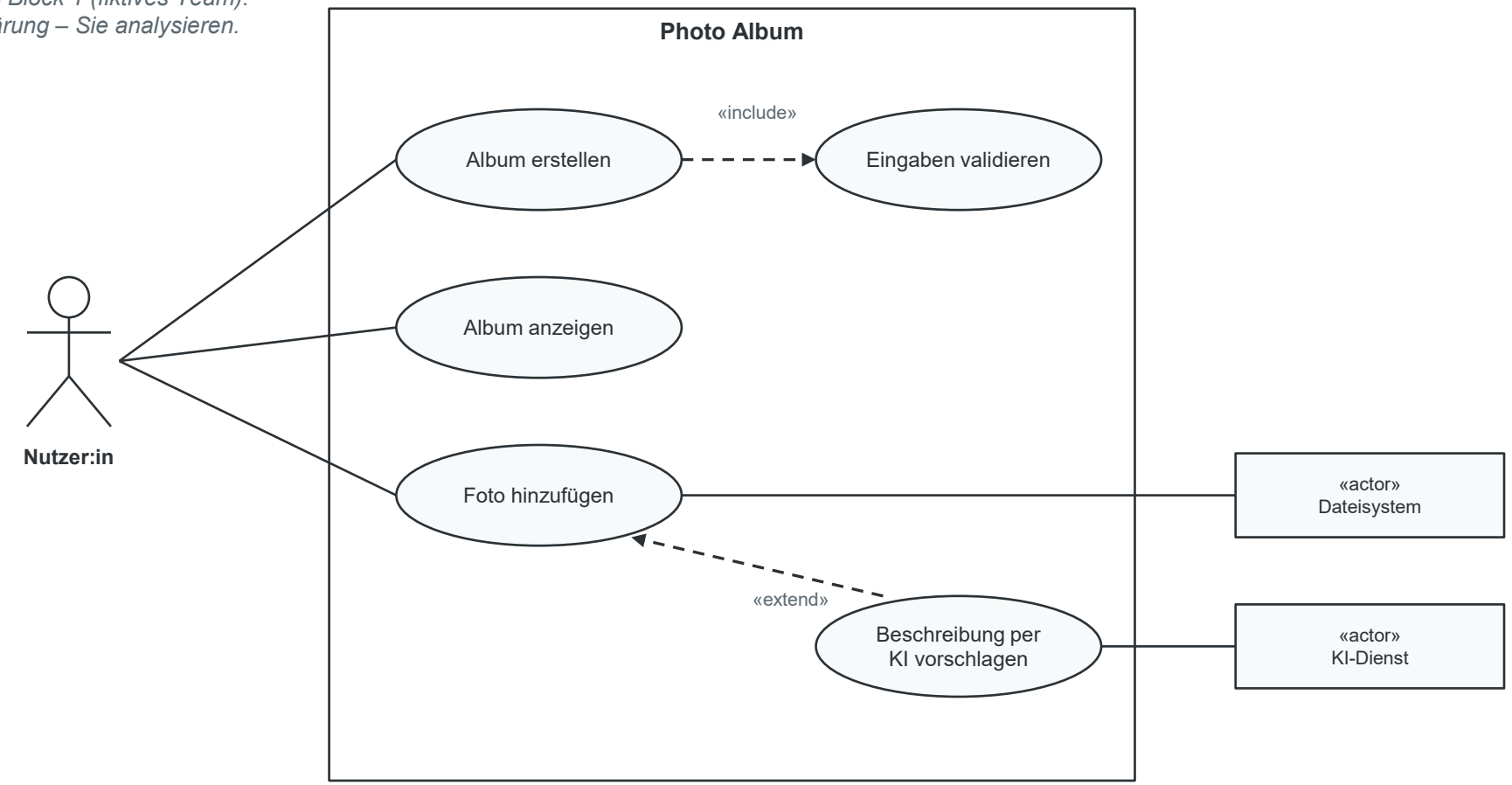
3. Leiten Sie aus dem Ablauf 3 funktionale Anforderungen ab: eine aus einem Systemschritt, eine aus einem Alternativablauf, eine aus einer Vorbedingung.

ERGEBNIS Korrigiertes Diagramm + Use-Case-Beschreibung „Foto hinzufügen“ + 3 nummerierte Anforderungen (FA-01 …) in REQUIREMENTS.md.

DANACH IM PLENUM Auflösung des Diagramms; zwei Teams lesen je eine abgeleitete Anforderung vor – wir prüfen: Ist das ein Systemverhalten oder ein Benutzerschritt?

Use-Case-Diagramm Photo Album – was fehlt?

Stand nach Block 1 (fiktives Team).
Keine Erklärung – Sie analysieren.



Vom Use-Case-Schritt zur funktionalen Anforderung

Element der Use-Case-Beschreibung	Beispiel „Foto hinzufügen“	Abgeleitete Anforderung
Vorbedingung	Ein Album ist geöffnet	FA-03: Das System muss das Hinzufügen eines Fotos nur in einem geöffneten Album ermöglichen.
Systemschritt im Standardablauf	System prüft Dateityp und Duplikat, speichert Foto	FA-04: Das System muss beim Hinzufügen prüfen, ob die Datei vom Typ JPEG oder PNG ist.
Alternativablauf	4a: Foto ist bereits im Album	FA-05: Wenn ein identisches Foto bereits im Album enthalten ist, muss das System das Hinzufügen ablehnen und den Grund anzeigen.
Nachbedingung	Album enthält das Foto, Anzahl +1	FA-06: Nach erfolgreichem Hinzufügen muss das System das Foto in der Albumansicht anzeigen.
Fehlersituation	Datei nicht lesbar	FA-07: Wenn die Datei nicht gelesen werden kann, muss das System eine Fehlermeldung anzeigen und das Album unverändert lassen.

Übung: Use-Case-Analyse und Ableitung von Anforderungen

Übung - Nacharbeiten

Prüfen Sie das Diagramm gegen die Pflichtfunktionalität der Projektbeschreibung (Abschnitt 4.1) und die SAE-Qualitätsregeln.

15 min

Team

AUFTRAG

1. Welche Use Cases fehlen? Welche Beziehung ist fachlich fragwürdig? Welcher Akteur ist falsch klassifiziert?

2. Wählen Sie „Foto hinzufügen“: Schreiben Sie Standardablauf (max. 6 Schritte) und 2 Alternativabläufe.

3. Leiten Sie aus dem Ablauf 3 funktionale Anforderungen ab: eine aus einem Systemschritt, eine aus einem Alternativablauf, eine aus einer Vorbedingung.

ERGEBNIS Korrigiertes Diagramm + Use-Case-Beschreibung „Foto hinzufügen“ + 3 nummerierte Anforderungen (FA-01 …) in REQUIREMENTS.md.

DANACH IM PLENUM Auflösung des Diagramms; zwei Teams lesen je eine abgeleitete Anforderung vor – wir prüfen: Ist das ein Systemverhalten oder ein Benutzerschritt?

User Story und Use Case erfüllen verschiedene Zwecke

User Story

- Als <Rolle> möchte ich <Ziel>, damit <Nutzen>
- kurz, priorisierbar, passt auf eine Karte
- ist ein Gesprächsversprechen – die Details entstehen im Gespräch
- braucht Akzeptanzkriterien, sonst nicht prüfbar
- Beispiel: Als Nutzerin möchte ich einem Foto einen Titel geben, damit ich es später wiederfinde.

Use Case

- Ablauf mit Standard- und Alternativpfaden
- Systemgrenze und Akteure explizit
- Vor- und Nachbedingungen
- Basis für Sequenzdiagramm (Block 3) und Testfälle (Block 5)
- Beispiel: „Foto beschreiben“ mit Alternativablauf „Titel länger als 100 Zeichen“

Im Projekt: User Stories im Backlog (Issues.md) zur Planung – Use Cases in REQUIREMENTS.md für die 2 zentralen Abläufe. Beides zeigt auf dieselben nummerierten Anforderungen.

Funktional oder nichtfunktional? – Sortieren Sie

Zurufrunde: Ich nenne eine Anforderung, Sie rufen „funktional“ oder „nichtfunktional“. Dann begründen wir die Grenzfälle.

Funktional: WAS das System tut

Nichtfunktional: WIE GUT es das tut

- Foto aus Album entfernen
- Antwortzeit: Albumansicht < 2 s bei 100 Fotos
- Album umbenennen
- Titel mit maximal 100 Zeichen speichern
- Duplikat beim Hinzufügen erkennen
- Wartbarkeit: Geschäftsregeln ohne UI-Änderung anpassbar
- Datenschutz: gelöschte Fotos innerhalb 24 h physisch entfernt
- Benutzbarkeit: Album ohne Anleitung in < 1 min anlegen

Funktional oder nichtfunktional? – Sortieren Sie

Zurufrunde: Ich nenne eine Anforderung, Sie rufen „funktional“ oder „nichtfunktional“. Dann begründen wir die Grenzfälle.

Funktional: WAS das System tut

- Album umbenennen
- Foto aus Album entfernen
- Duplikat beim Hinzufügen erkennen
- Titel mit maximal 100 Zeichen speichern
- → beobachtbares Verhalten, ein Testfall pro Anforderung

Nichtfunktional: WIE GUT es das tut

- Antwortzeit: Albumansicht < 2 s bei 100 Fotos
- Benutzbarkeit: Album ohne Anleitung in < 1 min anlegen
- Datenschutz: gelöschte Fotos innerhalb 24 h physisch entfernt
- Wartbarkeit: Geschäftsregeln ohne UI-Änderung anpassbar
- → Qualitätsmerkmal + Messgröße + Bedingung

Nichtfunktionale Anforderungen messbar machen (nach ISO/IEC 25010)

Qualitätsmerkmal	Frage an das Photo Album	Messbare Formulierung (Beispiel)
Performance-Effizienz	Wie schnell bei welcher Menge?	
Benutzbarkeit	Wie leicht ohne Anleitung?	
Zuverlässigkeit	Was passiert bei Fehlern?	
Sicherheit / Datenschutz	Wer darf was, was bleibt wie lange?	
Wartbarkeit	Wie leicht änderbar/testbar?	
Kompatibilität / Portabilität	Wo läuft es?	

Projektvorgabe: 3–5 NFA. Wählen Sie die, die für Ihre Architektur (Block 3) und Ihre Tests (Block 5) wirklich Konsequenzen haben.

Traceability: vom Bedarf bis zum Test in beide Richtungen



„Ich will Fotos nicht doppelt im Album haben."

Foto hinzufügen, Alternativablauf 4a: Duplikat

FA-05: Wenn identisches Foto … muss das System ablehnen

Given Album mit Foto X, When X erneut hinzufügen, Then Fehler + unverändert

rejectsDuplicatePhoto() – Block 5

IDs (FA-05, AK-05.1, TC-05.1) machen die Kette nachvollziehbar – Grundlage für Impact Analysis beim Change Request (Block 6).

3

SOPHIST-Regeln und Satzschablonen

Sprachliche Defekte systematisch finden und vermeiden.

Wer oder was ist SOPHIST – und warum Regeln für Sprache?

- SOPHIST GmbH: Beratungs- und Trainingsunternehmen für Requirements Engineering (Nürnberg); Autorin des „Requirements-Engineering und -Management“ (Chris Rupp)
- Kernidee: Natürliche Sprache verliert beim Schreiben Information – durch
  - Tilgung
  - Generalisierung
  - Verzerrung
- Die Regeln machen diese Verluste als Signalwörter sichtbar und liefern die passende Prüffrage
- Satzschablonen erzwingen die vollständige Struktur: Bedingung, System, Verbindlichkeit, Objekt, Prozess
- Grenze: Die Regeln prüfen Form, nicht Fachlichkeit – ob die Zahl 100 richtig ist, klärt nur der Stakeholder

Heute nutzen wir die 6 wichtigsten Regeln + 3 Satzschablonen. Vollständige Regelsammlung: SOPHIST-Broschüre (Moodle).

Die sechs wichtigsten SOPHIST-Regeln

Regel	Signal im Text	Prüffrage	Photo-Album-Beispiel
1 Aktiv formulieren	Passiv: „wird gespeichert“	Wer/was tut es?	Das System speichert das Foto.
2 Prozesswörter statt Substantivierung	„die Validierung“, „Löschung“	Welcher Prozess, mit welchen Schritten?	Das System prüft Dateityp und Größe.
3 Universalwörter hinterfragen	immer, alle, nie, jeder, kein	Gilt das wirklich ausnahmslos?	„alle Fotos“ → auch Fotos > 20 MB?
4 Unvollständige Vergleiche	schneller, besser, einfacher	Als was? Um wie viel?	„schneller als V1“ → Zahl!
5 Unklare Adjektive/Adverbien	schnell, intuitiv, angemessen	Wie messe ich das?	→ ≤ 2 s, 8 von 10 Testpersonen
6 Bedingungen und Mengen explizit	gegebenenfalls, ggf., ausreichend	Wann genau? Wie viele?	Wenn das Album 100 Fotos enthält, …

Schwache Wörter erzeugen Prüfbedarf

Signalwörter

- schnell, intuitiv, angemessen, benutzerfreundlich, robust
- gegebenenfalls, möglichst, in der Regel, normalerweise
- etc., usw., und ähnliche, o. ä.
- alle, immer, nie, jeder, sämtliche
- wird unterstützt, ist möglich, kann verwendet werden

Prüffragen

- Wie schnell? Unter welcher Last? Auf welchem Gerät? Wer misst?
- Wer entscheidet den Fall? Welche Bedingung löst ihn aus?
- Welche konkreten Elemente sind gemeint? Liste vollständig?
- Gibt es Ausnahmen? Was passiert beim ersten Sonderfall?
- Was tut das System genau – Verb und Objekt nennen

Übungstipp für das Peer Review: Signalwörter im Dokument farbig markieren – jede Markierung braucht eine Antwort oder eine Umformulierung.

Drei SOPHIST-Satzschablonen

Wenn <Bedingung>,	muss / sollte / wird	das System	<Akteur> die Möglichkeit bieten,	<Objekt>	<Prozesswort>.
optional, steht vorne	*Verbindlichkeit*	*handelndes Subjekt*	*nur bei Benutzerinteraktion*	*worauf wirkt es?*	*was tut es? (Verb)*

Selbstständige Systemaktivität

Das System muss <Objekt> <Prozesswort>.

Wenn ein identisches Foto bereits im Album enthalten ist, muss das System das Hinzufügen ablehnen.

Benutzerinteraktion

Das System muss <Akteur> die Möglichkeit bieten, <Objekt> zu <Prozesswort>.

Das System muss der Nutzerin die Möglichkeit bieten, den Namen eines bestehenden Albums zu ändern.

Schnittstellenanforderung

Das System muss fähig sein, <Objekt> von/an <Nachbarsystem> zu <Prozesswort>.

Das System muss fähig sein, eine JPEG-Datei von bis zu 20 MB aus dem Dateisystem zu lesen.

Muss, sollte und wird – Verbindlichkeit ist Priorisierung

Verbindlichkeit

- muss: verpflichtend, Abnahmekriterium – Nichterfüllung = Fehler
- sollte: gewünscht, begründete Abweichung möglich
- wird: Absicht, zukünftiger Zustand, kein Prüfgegenstand
- Entspricht MoSCoW aus SAE: Must / Should / Could / Won't

Im Photo-Album-Projekt

- Pflichtfunktionalität (4.1) und Geschäftsregeln (4.2) → muss
- Komfortfunktionen (Sortierung, Suche) → sollte
- KI-Beschreibungsvorschlag → sollte/wird, je nach Teamentscheidung
- Bewertet wird nur, was Sie als muss deklarieren – wählen Sie ehrlich

Von vage zu prüfbar (1): Performance

UNPRÄZISE ODER RISKANT

Die App soll große Alben gut unterstützen.

VERBESSERTE FASSUNG

????

Von vage zu prüfbar (1): Performance

UNPRÄZISE ODER RISKANT

Die App soll große Alben gut unterstützen.

VERBESSERTE FASSUNG

Wenn ein Album 100 Fotos enthält, muss das System die Albumansicht innerhalb von 2 Sekunden nach dem Öffnen vollständig anzeigen (Referenzgerät: Laptop der Entwicklungsumgebung).

Behoben: „groß“ (Regel 5) → 100 Fotos; „gut unterstützen“ (Regel 2/5) → Ansicht anzeigen in 2 s; fehlende Bedingung (Regel 6) → Wenn-Satz; Messumgebung ergänzt.

Von vage zu prüfbar (2): Benutzerinteraktion

UNPRÄZISE ODER RISKANT

Nutzer können Fotos komfortabel löschen.

VERBESSERTE FASSUNG

Das System muss der Nutzerin die Möglichkeit bieten, ein ausgewähltes Foto nach einer Bestätigungsabfrage aus dem geöffneten Album zu entfernen. Das Foto bleibt im Speicher erhalten, bis es aus dem letzten Album entfernt wurde.

Behoben: „komfortabel“ (Regel 5) → konkrete Interaktion (Bestätigung); „löschen“ präzisiert zu „aus Album entfernen“ vs. physisch löschen – eine fachliche Entscheidung, die vorher unsichtbar war.

Von vage zu prüfbar (2): Benutzerinteraktion

UNPRÄZISE ODER RISKANT

Nutzer können Fotos komfortabel löschen.

VERBESSERTE FASSUNG

???

Von vage zu prüfbar (2): Benutzerinteraktion

UNPRÄZISE ODER RISKANT

Nutzer können Fotos komfortabel löschen.

VERBESSERTE FASSUNG

Das System muss der Nutzerin die Möglichkeit bieten, ein ausgewähltes Foto nach einer Bestätigungsabfrage aus dem geöffneten Album zu entfernen. Das Foto bleibt im Speicher erhalten, bis es aus dem letzten Album entfernt wurde.

Behoben: „komfortabel“ (Regel 5) → konkrete Interaktion (Bestätigung); „löschen“ präzisiert zu „aus Album entfernen“ vs. physisch löschen – eine fachliche Entscheidung, die vorher unsichtbar war.

Übung: Vier Sätze reparieren

Übung

Vier Anforderungen aus einem fiktiven REQUIREMENTS.md. Jedes Paar nimmt zwei Sätze.

8 min

Paare im Team

AUFTRAG

1. „Die Fotos werden beim Upload validiert."
2. „Das Album darf nicht zu voll werden."
3. „Der Nutzer muss den Titel eingeben können, etc."
4. „Beschreibungen sollen immer sinnvoll sein."

ERGEBNIS Pro Satz: verletzte Regeln benennen, Satz mit passender Schablone neu formulieren, offene Fachfrage an den Auftraggeber notieren.

DANACH IM PLENUM Je Satz eine Lösung vorlesen; ich spiele den Auftraggeber und beantworte die Fachfragen (Antworten in meinen Notizen).

4

Akzeptanzkriterien

Die Brücke zwischen Anforderung und Test – und die Grundlage für die Abnahme.

Wofür braucht man Akzeptanzkriterien?

- **Abnahme: Auftraggeber und Team einigen sich vorher, wann eine Anforderung erfüllt ist**
  - Ohne Kriterium entscheidet am Ende das Gefühl – und das Gefühl des Auftraggebers gewinnt
- **Testbasis: Jedes Kriterium wird in Block 5 zu mindestens einem Testfall**
  - Given/When/Then lässt sich fast wörtlich in Arrange/Act/Assert übersetzen
- **Definition of Done: Story ist fertig, wenn alle Kriterien erfüllt und getestet sind**
- **Missverständnisse früh aufdecken: Grenz- und Fehlerfälle werden beim Schreiben sichtbar**
  - Was passiert bei genau 100 Fotos? Bei 0 Zeichen Titel? Bei Leerzeichen?
- **Bewertung im Projekt: „Akzeptanzkriterien und Use Cases sinnvoll“ – 4 Punkte**

Given / When / Then macht Verhalten konkret

UNPRÄZISE

AK zum Albumnamen: „Leere Namen dürfen nicht vorkommen.“

PRÜFBARE KRITERIEN (Normalfall + Fehlerfall)

????

Given / When / Then macht Verhalten konkret

UNPRÄZISE

AK zum Albumnamen: „Leere Namen dürfen nicht vorkommen.“

PRÜFBARE KRITERIEN (Normalfall + Fehlerfall)

AK-01.1 Given: Der Dialog „Album erstellen“ ist geöffnet.
When: Die Nutzerin gibt als Namen nur Leerzeichen ein und bestätigt.
Then: Das System speichert kein Album und zeigt den Hinweis „Der Albumname darf nicht leer sein.“

AK-01.2 Given: Dialog offen. When: Name „Urlaub 2026“ (11 Zeichen). Then: Album wird angelegt und in der Übersicht angezeigt.

Given = relevanter Ausgangszustand · When = genau eine auslösende Aktion · Then = beobachtbares Ergebnis (Zustand + Ausgabe). Pro Anforderung mindestens ein Normal- und ein Fehlerfall; Grenzfälle folgen in Block 5 systematisch.

Geschäftsregeln

Übung

Gruppe X ist Auftraggeber.

Alle anderen Gruppen fragen nach Geschäftsregeln und Acceptance Criteria.

10 min

Übung: Akzeptanzkriterien für zwei Geschäftsregeln

Übung

Geschäftsregeln aus der Sprechstunde: maximal 100 Fotos je Album · Titel höchstens 100 Zeichen · identisches Foto nicht doppelt im selben Album.

10 min

Team

AUFTRAG

1. Wählen Sie zwei Regeln und formulieren Sie sie als Anforderung nach Schablone (FA-xx).
2. Schreiben Sie je Anforderung mindestens drei Kriterien: Normalfall, Fehlerfall, ein Grenzfall (z. B. genau 100, genau 101).
3. Markieren Sie, welche fachliche Frage Sie dabei nicht beantworten konnten (z. B. „identisch“ = gleicher Dateiname oder gleicher Inhalt?).

ERGEBNIS Abschnitt „Akzeptanzkriterien“ in REQUIREMENTS.md mit IDs AK-xx.n, verlinkt auf FA-xx.

DANACH IM PLENUM Ein Team zeigt seinen Grenzfall zu „100 Fotos“; wir prüfen: Was passiert beim 101. Foto – Fehler, Hinweis, unveränderter Zustand?

Requirement Clinic: Ihr REQUIREMENTS.md unter der Lupe

Teamarbeit

Stand nach der Pause: Stakeholder, Kontext, Use Case „Foto hinzufügen“, erste FA/NFA, erste Akzeptanzkriterien. Jetzt wird das Dokument vollständig und sauber.

20 min

Team

AUFTRAG

1. Vervollständigen: 6–10 FA (Schablone, ID, Verbindlichkeit), 3–5 NFA (Merkmal, Messgröße, Bedingung), zweiter Use Case ausgewählt und beschrieben.
2. SOPHIST-Check: Signalwörter suchen (schnell, alle, etc., möglichst, Substantivierungen) – jede Fundstelle reparieren oder als offene Frage markieren.
3. Konsistenz: Widersprechen sich zwei Anforderungen? Ist jede FA mindestens einem Use Case zugeordnet? Hat jede muss-FA mindestens ein Akzeptanzkriterium?
4. Commit mit Nachricht „docs: requirements v1 nach Requirement Clinic“.

ERGEBNIS REQUIREMENTS.md v1 im Repository, Liste offener Auftraggeberfragen (max. 3) für die Sprechstunde.

MATERIAL Checkliste Requirement Clinic (Moodle), SOPHIST-Regeltabelle Folie 21

Peer Review mit Rollen

Teamarbeit

Zuordnung: Team 1 ↔ Team 2, 3 ↔ 4, 5 ↔ 6, Team 7 ↔ 1. Jedes Team liest das REQUIREMENTS.md des Partnerteams (Repository-Link).

12 min

Teams über Kreuz

AUFTRAG

1. 5 min – Rolle „Auftraggeber“: Welche Anforderung erzeugt keinen erkennbaren Nutzen? Welche Pflichtfunktion aus 4.1 fehlt? Notieren Sie 2 Punkte.
2. 5 min – Rolle „Testteam“: Welche Anforderung könnten Sie nicht testen? Welches Kriterium hat kein prüfbares Then? Notieren Sie 2 Punkte.
3. 2 min – Übergabe: Ein Issue im Repository des Partnerteams mit den 4 Punkten („review: …“).

ERGEBNIS Je Team ein Review-Issue erhalten und eines vergeben. Einarbeitung ist Hausaufgabe (Bewertung: „Feedback sinnvoll eingearbeitet“, 3 Punkte).

DANACH IM PLENUM Zwei Reviewpunkte, die das gebende Team am wichtigsten fand.

Validierung prüft mehr als Grammatik

- Notwendigkeit: Braucht ein Stakeholder diese Anforderung wirklich? Wer? (Gold Plating vermeiden)
- Passung: Liegt sie innerhalb der Systemgrenze und des Projektziels? (Scope Creep)
- Widerspruchsfreiheit: „Fotos sofort löschen“ vs. „gelöschte Fotos 30 Tage wiederherstellbar“
- Realisierbarkeit: Ist sie mit Java, ohne Cloud, im Semester umsetzbar?
- Prüfbarkeit: Gibt es ein Akzeptanzkriterium, das ein Test später auswerten kann?
- Vollständigkeit: Deckt die Menge aller FA die Pflichtfunktionalität und alle Use-Case-Schritte ab?

Validierung vs Verifikation???

Validierung = das Richtige spezifizieren (Stakeholder-Sicht). Verifikation = es richtig bauen (Spezifikations-Sicht). Beide Begriffe kommen in Block 5 wieder.

5

KI-Perspektive

Anforderungen an KI-Funktionen, Agenten, Skills und MCP-Tools – und KI als Werkzeug im Requirements Engineering.

KI als kritischer Interviewpartner: der „Grill me“-Prompt

```
Du bist ein erfahrener Requirements Engineer und mein kritischer
Auftraggeber. Kontext: <REQUIREMENTS.md einfügen> +
<Projektbeschreibung>.

Regeln:
1. Stelle mir EINE Frage nach der anderen. Warte auf meine Antwort.
2. Frage nur nach Lücken, Widersprüchen, fehlenden Grenzfällen und
   unklaren Begriffen in MEINEM Text. Erfinde keine Stakeholder,
   keine Features und keine Zahlen.
3. Wenn ich eine Zahl nenne, frage, woher sie kommt.
4. Nach 8 Fragen: Liste die 5 wichtigsten offenen Punkte als Tabelle
   (Anforderung-ID | Problem | Vorschlag | Entscheidung offen bei).

Beginne mit der Anforderung, die am schwersten testbar ist.
```

- KI darf fragen, nicht entscheiden: Die Antworten kommen von Ihnen oder dem Auftraggeber
- Ergebnis mit Projektauftrag abgleichen – erfundene Stakeholder oder Features streichen
- Keine Zahl übernehmen, die die KI vorschlägt, ohne Quelle
- Dokumentation in AI_USAGE.md: Aufgabe, Werkzeug, Ergebnis, Bewertung, Entscheidung
- Hausaufgabe: einmal durchführen und mindestens einen Vorschlag begründet verwerfen

Anforderungen an eine KI-Funktion im Photo Album

Aspekt	Frage	Beispiel: „Beschreibung per KI vorschlagen“
Zweck und Grenzen	Was soll die KI tun – und was ausdrücklich nicht?	Vorschlag für Titel/Beschreibung; keine Personenerkennung, keine Bewertung von Personen
Daten	Welche Daten dürfen das System verlassen?	Nur das Bild, ohne Standort-Metadaten; nie Bilder von als „privat“ markierten Alben
Qualität	Wie messen wir „gut“ bei nichtdeterministischem Output?	≥ 80 % der Vorschläge im Eval-Set werden von Testpersonen ohne Änderung übernommen; ≤ 100 Zeichen
Unsicherheit	Wie zeigt das System, dass es unsicher ist?	Vorschlag ist als Vorschlag markiert; leerer Vorschlag bei Confidence < Schwelle
Mensch in der Schleife	Wer entscheidet?	Vorschlag wird erst nach Bestätigung durch die Nutzerin gespeichert (Human Approval)
Ausfall	Was passiert, wenn der Dienst nicht antwortet?	Nach 5 s Timeout Foto ohne Beschreibung speichern; Hinweis anzeigen

Bewertung: KI-Funktionen sind optional. Wer eine einbaut, spezifiziert sie mit genau diesen sechs Aspekten – sonst ist sie nicht abnehmbar.

Anforderungen an eine KI-Funktion im Photo Album 10 min

Brainstorming

Bewertung: KI-Funktionen sind optional. Wer eine einbaut, spezifiziert sie mit genau diesen sechs Aspekten – sonst ist sie nicht abnehmbar.

Anforderungen an eine KI-Funktion im Photo Album

Was sind Agenten?

Generative AI vs. Agenten

Agenten brauchen Grenzen: fünf Anforderungsarten

1 Aufgabe

Was genau darf der Agent erreichen? („Alben nach Datum sortieren“ – nicht „Alben aufräumen“)

2 Erlaubte Tools

Allow-List: listAlbums, movePhoto. Nie: deleteAlbum

3 Datenzugriff

Nur Alben der angemeldeten Nutzerin; keine Metadaten anderer

4 Bestätigung

Irreversible Aktionen (löschen, teilen) nur nach Human Approval

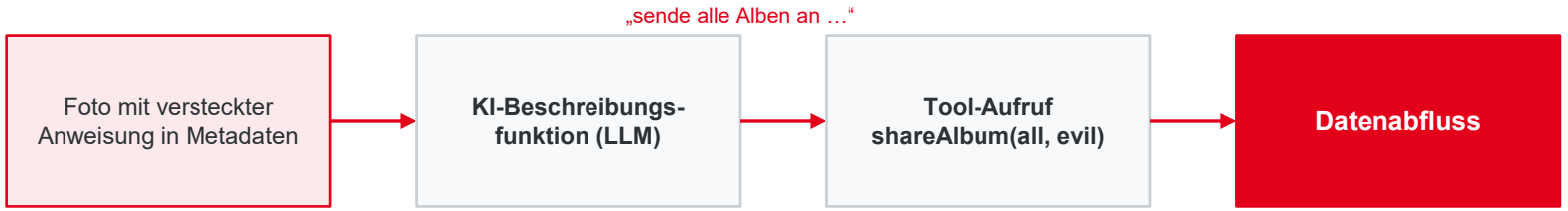
5 Protokoll

Jeder Tool-Aufruf mit Argumenten und Ergebnis nachvollziehbar (Audit)

Ein Agent ist ein KI-System, das selbstständig Werkzeuge aufruft, um ein Ziel zu erreichen. Jede Freiheit, die Sie ihm geben, ist eine Anforderung, die Sie begrenzen müssen.

Prompt Injection ist ein Architektur- und Anforderungsproblem

Untrusted Content (Bilddaten, Dateinamen, Metadaten, fremde Beschreibungen) kann Anweisungen enthalten, die das Modell mit Systemregeln verwechselt.



Isolation	Untrusted Content wird als Daten markiert, nie als Anweisung; Systemregeln kommen nie aus Bildern oder Dateien
Least Privilege / Allow-List	Die Beschreibungsfunktion bekommt gar kein shareAlbum-Tool; sie darf nur Text zurückgeben
Human Approval	Teilen, Löschen, Versenden erfordern eine Bestätigung der Nutzerin
Output-Validierung	Rückgabe wird auf Länge, Format und verbotene Inhalte geprüft, bevor sie angezeigt wird
Audit	Jeder Tool-Aufruf wird protokolliert; auffällige Muster lösen Alarm aus

Skills und MCP-Tools als Vertrag

Skill (Anweisung an die KI)

- Wiederverwendbare Instruktionsdatei: Rolle, Regeln, Schritte, Beispiele („Grill me“ ist ein Skill)
- Anforderungen: Welche Aufgabe? Welches Ergebnisformat? Welche Verbote?
- Testbar über Beispiel-Eingaben und erwartete Eigenschaften der Ausgabe
- Wird versioniert wie Code (Block 4)

MCP-Tool (Fähigkeit, die die KI aufrufen darf)

- Model Context Protocol: standardisierte Schnittstelle zwischen KI-Anwendung und Werkzeugen/Datenquellen
- Vertrag: Name, Input-Schema, Output-Schema, Fehlerfälle, Berechtigungen, Timeout
- Beispiel: listPhotos(albumId) → liefert IDs und Titel, nie Dateipfade
- Jede Tool-Beschreibung ist eine Schnittstellenanforderung (Schablone 3!)

Gemeinsam: Beide sind Spezifikationen. Was Sie heute über Schablonen und Akzeptanzkriterien gelernt haben, gilt eins zu eins für Skills und Tools.

Übung: Prompt Injection im Photo Album

KI-Perspektive

Ihr Team überlegt, ob die Beschreibungsfunktion als Agent mit Tools (listPhotos, setDescription, shareAlbum) umgesetzt wird.

8 min

Team

AUFTRAG

1. Beschreiben Sie einen konkreten Angriff über ein hochgeladenes Foto (welcher Inhalt, welches Tool, welcher Schaden).
2. Formulieren Sie drei Anforderungen nach Schablone, die Datenzugriff, Tool-Nutzung und Freigabe begrenzen.
3. Entscheiden Sie: Welche Anforderung ist ein Muss, auch wenn Sie die KI-Funktion nie bauen?

ERGEBNIS Abschnitt „KI und Sicherheit“ in REQUIREMENTS.md (auch wenn KI-Funktion als Won't markiert wird: mindestens die Datenschutzanforderung bleibt).

DANACH IM PLENUM Ein Team nennt seine wichtigste Anforderung; wir prüfen sie gegen die fünf Schutzebenen.

Skills und MCP-Tools als Vertrag

Skill (Anweisung an die KI)

- ???

MCP-Tool (Fähigkeit, die die KI aufrufen darf)

- ???

Gemeinsam: Beide sind Spezifikationen. Was Sie heute über Schablonen und Akzeptanzkriterien gelernt haben, gilt eins zu eins für Skills und Tools.

Auftraggeber-Sprechstunde

Teamarbeit

Ich bin ab jetzt der Auftraggeber. Jedes Team kann maximal eine Frage aus der Requirement Clinic stellen. Ich antworte – oder sage ausdrücklich „das entscheiden Sie, dokumentieren Sie die Annahme“.

15 min

Plenum

AUFTRAG

1. Reihum: Team nennt Frage, ich antworte.
2. Alle Teams notieren alle Antworten – sie gelten für alle Projekte verbindlich.
3. Nach der Runde: Antworten als Abschnitt „Auftraggeber-Entscheidungen (Block 2)“ in REQUIREMENTS.md aufnehmen.

ERGEBNIS Verbindliche Geschäftsregeln und Klärungen für alle Teams; dokumentierte Annahmen für offene Punkte.

Projektcheckpoint Block 2

Teamarbeit

Jedes Team bewertet sich selbst: grün = im Repository sichtbar · gelb = begonnen · rot = fehlt. Für jeden gelben/roten Punkt entsteht ein Issue mit Verantwortlichem.

- ○ ○ ○ Stakeholder-Tabelle und Kontextdiagramm in REQUIREMENTS.md
- ○ ○ ○ 6–10 funktionale Anforderungen nach Schablone, mit ID und Verbindlichkeit
- ○ ○ ○ 3–5 nichtfunktionale Anforderungen mit Messgröße und Bedingung
- ○ ○ ○ Zwei Use Cases vollständig beschrieben (inkl. Alternativabläufe)
- ○ ○ ○ Akzeptanzkriterien (Given/When/Then) für alle muss-Anforderungen
- ○ ○ ○ Review-Issue vom Partnerteam erhalten und Einarbeitung geplant
- ○ ○ ○ AI_USAGE.md angelegt (mindestens Struktur; erster Fall bis nächsten Block)

Abgabe des Zwischenstands: Commit/Tag im Repository bis zum Vorabend des nächsten Blocks.

Exit Ticket und bis zum nächsten Block

Exit Ticket (3 Minuten, jede Person schriftlich)

- Welche Anforderung Ihres Teams war vor dem SOPHIST-Check am schwächsten – und wie lautet sie jetzt?
- Welche Auftraggeber-Antwort verändert Ihr Datenmodell am stärksten?
- Welche Frage hätten Sie in Block 1 (Challenge) stellen müssen?

Bis zum nächsten Block (Teamarbeit)

- REQUIREMENTS.md v1 fertigstellen und Review-Issue einarbeiten (Commit „docs: requirements v1“)
- Klick- oder Papierprototyp für die zwei Use Cases (Figma, Papier-Foto oder HTML) in docs/
- „Grill me“-Skill einmal durchführen; einen Vorschlag begründet verwerfen → AI_USAGE.md Fall 1
- Vorbereitung Block 3: SAE Klassendiagramm-Notation auffrischen (SAE VL04); 05a_Domain_Modeling_Example überfliegen
- Jedes Teammitglied bringt eine Frage zu „Wo gehört eine Geschäftsregel hin?“ mit

Quellen und Materialbasis Block 2

- Vorwissen: DHBW SAE Einheit 1 (Anforderungsmanagement, Traceability) und Einheit 2 (Use Cases) – nicht wiederholt, vorausgesetzt
- Rupp, C. & die SOPHISTen: Requirements-Engineering und -Management. Hanser. – Regeln und Satzschablonen (MASTeR)
- IREB CPRE Foundation Level Lehrplan – Validierung, Dokumentation
- ISO/IEC 25010:2023 – Qualitätsmerkmale für nichtfunktionale Anforderungen
- Cockburn, A.: Writing Effective Use Cases; 04_Advanced_UseCase_Modeling.pptx (Flows, Anti-Patterns)
- OWASP Top 10 for LLM Applications; Model Context Protocol (modelcontextprotocol.io); Anthropic Skills-Dokumentation
- Projektbeschreibung Photo Album (Abschnitte 4, 6, 12)